✅ Senior C# Code Review Checklist (.NET 10 + C#, март 2026)

1) Архитектура и дизайн системы (1–30)

1. Архитектура соответствует стилю проекта: Clean / Onion / Hexagonal / Modular Monolith / Microservices.
2. Domain не зависит от Infrastructure и UI.
3. Domain слой не содержит Entity Framework / HttpClient / файловых операций.
4. Бизнес-логика не находится в Controllers / Endpoints / HostedService.
5. Нет God-классов (Service который “делает всё”).
6. Модули имеют чёткие границы ответственности.
7. У каждого bounded context есть свой слой Domain.
8. Применяется CQRS только если это оправдано.
9. Нет смешивания CRUD и сложного домена без объяснения.
10. Используются Value Objects вместо примитивов (string, int как доменные типы).
11. Используются immutable модели где возможно (record, init, readonly).
12. Агрегаты защищают инварианты.
13. Нет “анемичной модели” если заявлен DDD.
14. Используются доменные события (Domain Events) где это уместно.
15. Нет утечки DTO/Entity наружу.
16. Применяются Application Services как слой сценариев (use-cases).
17. Infrastructure содержит интеграции (DB, Kafka, Redis, внешние API).
18. Применяется Dependency Inversion: зависимости направлены внутрь.
19. Нет service locator / static container.
20. Нет “магических” singletons, которые создаются вручную.
21. DI регистрации вынесены в extension-методы.
22. Нет логики в Program.cs (только wiring).
23. Конфигурация оформлена через Options Pattern.
24. Options валидируются при старте приложения.
25. Нет “общего проекта Utils”, который растёт бесконечно.
26. Используются Feature Folders или модульная структура.
27. В микросервисах соблюдается независимость схем БД.
28. Используются API contracts (OpenAPI/Swagger) как источник правды.
29. В коде нет циклических зависимостей проектов.
30. Архитектура документирована (README + диаграмма модулей).

---

2) Структура кода и организация файлов (31–55)

31. Один файл — одна ответственность.
32. Один класс — одна роль.

33. Нет файлов больше 500 строк без причины.
34. Классы не превышают ~300 строк.
35. Методы не превышают ~30–50 строк.
36. Методы не имеют 6+ параметров (использовать параметры-объект).
37. Вложенность if/for не глубже 3 уровней.
38. Используются file-scoped namespaces.
39. Используются primary constructors если это улучшает читаемость.
40. Части больших сущностей разбиваются через partial (только оправдано).
41. DTO не смешиваются с Entity.
42. Domain модели не содержат атрибутов сериализации.
43. Нет “универсальных” Helper классов.
44. Нет лишних интерфейсов ради “красоты”.
45. Интерфейсы создаются только при реальной необходимости.
46. Папки/проекты отражают архитектуру (не хаос).
47. Код разделён по слоям и модулям, а не по типам (Models/Services/Helpers).
48. Используются отдельные проекты: Domain, Application, Infrastructure, Api.
49. Тесты структурированы так же как прод-код.
50. Конфигурации/extension-методы сгруппированы логически.
51. Нет статических “глобальных” состояний.
52. Нет “общих” сущностей между микросервисами (shared domain).
53. Используется explicit visibility (internal/public осознанно).
54. Минимум public API.
55. Проект собирается без warnings.

---

3) Naming и смысловые имена (56–80)

56. Имена отражают бизнес-смысл.
57. Классы — существительные.
58. Методы — глаголы.
59. Async методы заканчиваются на Async.
60. CancellationToken параметр называется cancellationToken.
61. Переменные читаются как предложение.
62. Булевы имена читаются как вопрос (isEnabled, hasAccess).
63. Нет сокращений кроме общеизвестных (Id, Url, Http).
64. Нет “Manager”, “Helper”, “Service” без уточнения.
65. Названия не содержат деталей реализации (SqlUserRepository ❌).
66. Названия отражают уровень абстракции (Repository, Provider, Gateway).
67. Нет одинаковых имен с разным смыслом.
68. Нет слишком длинных имён.
69. Нет слишком коротких имён (a, b, tmp).
70. Переменные цикла i/j допустимы только локально.
71. Методы не называются DoWork, HandleSomething.
72. Используются общие правила .NET naming conventions.
73. Используются суффиксы: Handler, Validator, Middleware, Endpoint.
74. В тестах имена описывают сценарий.

75. Публичные API имеют ясные контракты.
76. В логах используются понятные названия событий.
77. Исключения имеют правильный нейминг (InvalidOrderStateException).
78. DTO различаются Request/Response.
79. Команды/запросы называются CreateOrderCommand / GetOrderQuery.
80. Модули имеют бизнес-названия, не технические.

---

4) Читаемость и Clean Code правила (81–115)

81. Код читается без комментариев.
82. Комментарии объясняют “почему”, а не “что”.
83. Нет закомментированного кода.
84. Нет TODO без задачи (issue/тикета).
85. Используются guard clauses вместо глубокой вложенности.
86. Используется early return.
87. Нет “магических чисел”.
88. Константы вынесены и названы.
89. Нет огромных switch на 10+ кейсов без стратегии.
90. Используются switch expressions и pattern matching.
91. LINQ не превращён в “головоломку”.
92. Нет LINQ-цепочек >3–4 операций.
93. Нет side-effects в LINQ.
94. Нет сложных тернарников.
95. Не используется dynamic без необходимости.
96. Сложные вычисления вынесены в отдельные методы.
97. Методы делают одну вещь.
98. Нет повторения одинаковых блоков кода.
99. Повторение убирается через abstraction или extension-методы.
100. Не злоупотребляют extension-методами (не превращать в “магический код”).
101. Используются современные возможности C# (records, init, pattern matching).
102. Используются collection expressions `[ ]` вместо new List.
103. Используются required-свойства если нужно enforce контракт.
104. Используются generic attributes где оправдано.
105. Используются nameof вместо строк.
106. Используются using declarations вместо using blocks где удобно.
107. Нет логики в property getter/setter.
108. Конструкторы не содержат сложной логики.
109. Используются явные типы если var ухудшает читаемость.
110. Используется var если тип очевиден.
111. Нет огромных файлов Program.cs.
112. Минимальные API endpoints читаются как сценарии.
113. Нет “скрытой магии” через reflection.
114. Код форматирован единообразно.
115. Включён .editorconfig и соблюдается.

5) Ошибки, исключения и устойчивость (116–145)

116. Исключения используются только для исключительных ситуаций.
117. Нет пустых catch.
118. Нет swallow exceptions.
119. Используется throw; а не throw ex;
120. Исключения типизированы (не Exception).
121. Нет throw new Exception("error").
122. Ошибки логируются структурировано.
123. StackTrace сохраняется.
124. Ошибки API не раскрывают внутренние детали.
125. API возвращает ProblemDetails (RFC 7807).
126. Используются корректные HTTP статус-коды.
127. Валидация входных данных происходит до выполнения логики.
128. Валидация не размазана по всему коду.
129. Используются Validation Pipeline / Filters / FluentValidation.
130. Retry используется только для transient ошибок.
131. Retry ограничен лимитом и backoff стратегией.
132. Используется Polly (или встроенные resilience pipeline).
133. Есть circuit breaker для внешних сервисов.
134. Нет infinite retry.
135. Таймауты прописаны для всех внешних вызовов.
136. Используется CancellationToken.
137. CancellationToken реально передаётся вниз по стеку.
138. Логируются критические сбои приложения.
139. Есть централизованный error handling middleware.
140. Есть глобальная стратегия обработки ошибок.
141. Логи не содержат PII/секретов.
142. В API нет “return null вместо ошибки”.
143. Ошибки домена выражены Result/Either (если принято в проекте).
144. Domain ошибки не превращаются в Exception без причины.
145. Ошибки не используются как flow control.

6) Async/Await и параллелизм (146–175)

146. Нет .Result и .Wait().
147. Нет deadlock-опасных конструкций.
148. Async используется только для I/O.
149. Нет лишнего Task.Run в серверном коде.
150. Async методы возвращают Task/Task<T>.
151. Нет async void кроме событий.
152. ValueTask применяется только при реальной выгоде.
153. Нет async методов без await.

154. Нет sync wrappers поверх async.
155. Нет блокировок lock внутри async кода.
156. Используются SemaphoreSlim при необходимости.
157. Используются Channels для очередей.
158. Используется Parallel.ForEachAsync осознанно.
159. Параллельность ограничена (MaxDegreeOfParallelism).
160. Нет race conditions при shared state.
161. Shared mutable state минимизирован.
162. Используется IAsyncEnumerable для streaming.
163. Нет лишних ConfigureAwait(false) в приложениях.
164. ConfigureAwait(false) используется в библиотеках.
165. CancellationToken применяется в Task.Delay/WaitAsync.
166. Нет бесконечных циклов без задержек.
167. BackgroundService корректно останавливается.
168. Нет “зависших” задач.
169. Task.WhenAll используется безопасно.
170. Обрабатываются AggregateException где нужно.
171. Потоки не создаются вручную без причины.
172. Используется PeriodicTimer вместо Timer.
173. Нет утечек таймеров.
174. Http запросы имеют timeout.
175. Используется HttpClientFactory.

---

7) Производительность и .NET 10 практики (176–215)

176. Нет лишних аллокаций в горячих местах.
177. Нет boxing/unboxing.
178. LINQ не используется в критичных циклах.
179. Нет повторных ToList/ToArray без смысла.
180. Нет двойных enumerations.
181. Используются Span/Memory если это оправдано.
182. Используется ReadOnlySpan<char> при парсинге.
183. Строки не конкатенируются в циклах (используется StringBuilder).
184. Используется ArrayPool при больших буферах.
185. Используется MemoryCache / DistributedCache при необходимости.
186. JSON сериализация настроена (System.Text.Json).
187. Используется source generation для System.Text.Json если есть высокая нагрузка.
188. Нет reflection в горячих местах.
189. Используются source generators вместо reflection.
190. Используется NativeAOT если нужно уменьшить старт/память.
191. Если включён trimming — код совместим (нет динамики без атрибутов).
192. Нет лишних логов в горячих местах.
193. Логи используют LogLevel корректно.
194. В EF Core нет N+1 запросов.

195. EF Core использует AsNoTracking где нужно.
196. EF Core запросы проецируются в DTO (Select).
197. Нет “Include всего подряд”.
198. Индексы в БД соответствуют запросам.
199. Нет огромных JSON ответов без pagination.
200. Используется streaming для больших данных.
201. Используется compression (gzip/brotli) если нужно.
202. Используется response caching где оправдано.
203. Используется pooling для DbContext через AddDbContextPool.
204. Используются batch операции вместо 1000 insert.
205. Нет лишнего использования dynamic.
206. Нет лишних allocations в лямбдах.
207. Замыкания не удерживают большие объекты.
208. Используются struct/record struct где нужно.
209. Нет больших struct > 32 байт.
210. Используются Frozen collections если нужны read-only быстрые lookup.
211. Нет лишнего thread blocking.
212. Есть метрики latency/allocations.
213. Используются BenchmarkDotNet для критичных частей.
214. Производительность тестируется нагрузкой.
215. Приложение стартует быстро (особенно если контейнер).

---

8) Работа с ресурсами и IDisposable (216–235)

216. Все IDisposable освобождаются.
217. Используется using/await using.
218. Нет утечек сокетов.
219. HttpClient не создаётся вручную в каждом запросе.
220. HttpClientFactory используется правильно.
221. Потоки закрываются корректно.
222. Нет утечек file handles.
223. DbContext не живёт дольше запроса.
224. Нет открытых транзакций.
225. Connection не держится дольше необходимого.
226. Нет блокировок на долгих операциях.
227. CancellationToken применяется везде.
228. PeriodicTimer используется вместо Timer.
229. BackgroundService корректно dispose.
230. Нет бесконечных ретраев.
231. Нет бесконечного роста коллекций в памяти.
232. Логи не растут в памяти (buffered logging).
233. Используется IHostedService правильно.
234. Очереди сообщений не переполняются без backpressure.
235. Нет утечек unmanaged ресурсов.

9) Безопасность (236–275)

236. Нет SQL инъекций.
237. Все запросы параметризованы.
238. Нет хранения секретов в коде.
239. Используется Secret Manager / Vault / Key Vault.
240. Конфиги не содержат пароли в репозитории.
241. Пароли не логируются.
242. В логах нет JWT/refresh токенов.
243. Используется HTTPS везде.
244. Включён HSTS.
245. Включён безопасный CORS (не AllowAnyOrigin).
246. JWT подписан и валиден.
247. JWT имеет exp, aud, iss.
248. Refresh token хранится безопасно.
249. Не хранить токены в localStorage (если web).
250. Cookies имеют Secure + HttpOnly + SameSite.
251. Нет открытого Swagger в проде.
252. Swagger защищён авторизацией или выключен.
253. Health endpoints закрыты или ограничены.
254. Admin endpoints закрыты.
255. Rate limiting включён (.NET 10 built-in).
256. Есть защита от brute force.
257. Входные данные валидируются.
258. Нет доверия к данным клиента.
259. File upload ограничен по размеру.
260. Проверяется тип файла.
261. Нет path traversal.
262. Нет XSS.
263. Нет CSRF.
264. Нет SSRF.
265. Используются policies authorization.
266. RBAC/ABAC реализованы осознанно.
267. Используется hashing паролей (bcrypt/argon2).
268. Нет “самописного шифрования”.
269. Используются актуальные TLS версии.
270. Security headers настроены (CSP, X-Frame-Options).
271. Используются dependency vulnerability scanners.
272. Нет устаревших пакетов.
273. Включён audit зависимостей.
274. Все endpoints требуют auth если нужно.
275. Ошибки не раскрывают стек наружу.

10) API, контракт и интеграции (276–310)

276. API соответствует REST или выбранному стилю.
277. DTO не протекают внутрь Domain.
278. Версионирование API продумано.
279. Ошибки стандартизированы (ProblemDetails).
280. Swagger актуален и не врёт.
281. Документация endpoint-ов понятна.
282. Используется pagination для списков.
283. Используется фильтрация/сортировка корректно.
284. Есть idempotency для POST где нужно.
285. Внешние интеграции имеют timeout + retry.
286. Внешние интеграции имеют circuit breaker.
287. Обрабатываются ошибки сети.
288. Webhook обработчики проверяют подпись.
289. Внешние API вызовы логируются безопасно.
290. Есть correlation id / trace id.
291. Используется OpenTelemetry tracing.
292. Используется structured logging.
293. Используется Serilog / Microsoft logging best practices.
294. Логи не превращены в “Console.WriteLine”.
295. Интеграции не захардкожены в коде.
296. Конфиги интеграций вынесены в Options.
297. У интеграций есть интерфейс (Gateway/Client).
298. Есть контрактные тесты если важно.
299. JSON контракты стабильны.
300. Обрабатываются nullable поля.
301. Используется правильная кодировка.
302. Нет лишних sync вызовов.
303. API не возвращает Entity Framework entity напрямую.
304. В API нет циклических ссылок в JSON.
305. Не отдаётся лишняя информация.
306. Есть лимит на размер response.
307. Для больших файлов используется streaming.
308. Используется compression.
309. Response caching используется осознанно.
310. API устойчив к злоупотреблению.

---

11) База данных и EF Core (311–345)

311. Нет N+1 запросов.
312. Используется AsNoTracking для read-only.
313. Нет Include всего подряд.
314. Используется projection (Select DTO).
315. Запросы оптимизированы.

316. Есть индексы под реальные фильтры.
317. Миграции не ломают прод.
318. Миграции версионируются.
319. Нет логики миграции внутри приложения без контроля.
320. Транзакции используются осознанно.
321. Транзакции минимальны по времени.
322. Нет “SaveChanges в цикле”.
323. Нет “Select *”.
324. Нет загрузки огромных таблиц в память.
325. Есть paging.
326. Есть soft delete если требуется.
327. Есть concurrency token если нужен optimistic locking.
328. Нет ручного SQL без необходимости.
329. Raw SQL безопасен и параметризован.
330. Нет доменной логики в EF конфигурации.
331. Нет навигационных коллекций без смысла.
332. Нет ленивой загрузки без понимания последствий.
333. Используются Compiled Queries если нужно.
334. Используется DbContextPool если нужно.
335. Конфигурация EF вынесена из Program.cs.
336. Есть retry policy на DB соединение.
337. Есть timeout на DB операции.
338. База данных не используется как очередь.
339. Схема БД соответствует домену.
340. Нет “таблица на всё”.
341. В запросах нет тяжёлых вычислений.
342. Используются правильные типы колонок.
343. Нет хранения дат как string.
344. Время хранится в UTC.
345. Маппинг корректен и покрыт тестами.

---

12) Тесты, качество и DevOps (346–400)

346. Бизнес-логика покрыта unit тестами.
347. Есть интеграционные тесты для БД.
348. Есть тесты API (WebApplicationFactory).
349. Есть тесты безопасности (basic).
350. Есть тесты производительности (хотя бы smoke).
351. Тесты детерминированы.
352. Нет тестов, зависящих от времени без фиксации времени.
353. Используется TimeProvider (.NET).
354. Используются TestContainers где нужно.
355. Нет тестов, зависящих от внешнего API.
356. Есть мок/фейк для интеграций.
357. Есть snapshot тесты для DTO если нужно.

358. Нет отключённых тестов.
359. CI запускает тесты автоматически.
360. CI запускает линтеры.
361. Включены analyzers.
362. Включён nullable.
363. Включён TreatWarningsAsErrors.
364. Включён StyleCop/EditorConfig.
365. Используется SonarQube или аналог.
366. Используется dependency scanning.
367. Используется Dependabot/renovate.
368. Версии пакетов фиксируются.
369. Есть Dockerfile для сервисов.
370. Есть health checks.
371. Есть readiness/liveness probes.
372. Есть логирование структурировано.
373. Есть метрики (Prometheus/OpenTelemetry).
374. Есть distributed tracing.
375. Есть алерты на ошибки.
376. Логи коррелируются traceId.
377. Используются feature flags если нужно.
378. Есть rollback стратегия.
379. Есть миграции pipeline.
380. Infrastructure as Code применяется.
381. Есть мониторинг ресурсов.
382. Настроены limits/requests в Kubernetes (если используется).
383. Приложение не падает при временных сбоях.
384. Есть graceful shutdown.
385. Background jobs контролируются.
386. Очереди имеют retry policy.
387. Есть дедупликация сообщений.
388. Есть dead-letter queue.
389. Нет ручных деплоев в прод.
390. Есть окружения dev/stage/prod.
391. Настройки окружений не смешиваются.
392. Включены security checks в CI.
393. Используются pre-commit hooks.
394. PR проходит review.
395. Есть codeowners.
396. Есть release notes/версионирование.
397. Соблюдается semantic versioning.
398. Ветка main всегда стабильна.
399. Продукт собирается reproducible.
400. Все артефакты логируются и отслеживаются.

⚡ Мини-правила Clean Code (коротко, но жёстко)

- **Читаемость важнее компактности**
- **Сложность важнее длины**
- **Архитектура важнее синтаксиса**
- **Инварианты домена важнее DTO**
- **Слой Domain не знает ни о чём внешнем**
- **Код должен быть простым для поддержки, а не умным**

🔥 Топ-20 критических ошибок, которые нельзя пропускать в review

1. Бизнес-логика в контроллере/endpoint.
2. DbContext в Domain.
3. Отсутствие CancellationToken.
4. Retry без лимита.
5. HttpClient создаётся через new.
6. Логирование токенов/паролей.
7. AllowAnyOrigin в CORS.
8. SQL строки конкатенацией.
9. N+1 в EF Core.
10. SaveChanges в цикле.
11. .Result / .Wait().
12. Swallow exceptions.
13. throw ex.
14. Нет ProblemDetails.
15. Нет валидации входных данных.
16. DTO = Entity.
17. Shared mutable state в параллельных задачах.
18. Нет таймаутов у внешних API.
19. Отсутствие health checks.
20. Нет structured logging / traceId.